

Sprawozdanie z projektu symulatora UAR

sieci komputerowe

Skład Sekcji i Podział Pracy

- **Wojciech Kozłowski:**

Dokonanie weryfikacji i walidacji poprawności otrzymanej aplikacji. Poprawienie znalezionych uchybień, pomoc przy problemie utraty pakietów oraz zabezpieczeń mechanizmu ponownego łączenia.

- **Paweł Urbaniak:**

Opracowanie mechanizmów serializacji i deserializacji danych, implementacja rozproszonej architektury układu regulacji, projekt i logika trybu z obustronnym taktowaniem.

- **Krzysztof Rostek:**

Projekt ramek oraz ich implementacja, Zliczanie przesłanych pakietów oraz występujących opóźnień, Pomoc przy obsłudze przychodzących pakietów w trybie sieciowym

Protokół Komunikacyjny, Serializacja:

- **Protokół i Gniazda:** Komunikacja sieciowa została zrealizowana przy użyciu asynchronicznych socketów dostarczanych przez bibliotekę Qt.
- **Serializacja:** Zaimplementowaliśmy dwa rodzaje serializacji przesyłania danych: tekstową oraz binarną do wyboru w trakcie nawiązywania połączenia obiektu z regulatorem. Dodatkowo interfejs dostarcza statystyki wydajnościowe takie jak liczba pakietów na sekundę do porównania obu typów.
- **Zarządzanie pakietami:** Wymiana danych w symulacji opiera się na id kolejności próbek, co pozwala na precyzyjne śledzenie opóźnień transmisji oraz desynchronizacji instancji.
- **Wydajność symulacji:** Stan nadążania symulacji za interwałem jest wizualizowany w GUI przy pomocy zielonej/czerwonej lampki, który informuje nas czy system "wyrabia się" z obliczeniami.
- **Protokół komunikacyjny i struktura ramek:** Komunikacja w architekturze klient-serwer opiera się na protokole TCP. Zaimplementowaliśmy projekt ramek w którym każda ramka składa się z nagłówka określającego typ pakietu oraz pola danych serializowanych binarnie lub tekstowo. Każda ramka zawiera ID co pozwala śledzić desynchronizację. Komunikacja nie blokuje głównego wątku GUI dzięki wykorzystaniu architektury sterowanej zdarzeniami.
- **Mechanizmy serializacji:** Aplikacja wspiera dwa formaty serializacji, wybierane w interfejsie użytkownika. Podstawowy przesył cyklicznych

próbek pomiarowych wykorzystuje serializację binarną, co minimalizuje narzut transmisyjny. Jako alternatywa, dla ułatwienia analizy, dostępna jest serializacja tekstowa. Złożone struktury, takie jak pełna konfiguracja, przesyłane są wyłącznie w formacie JSON.

Wykrywanie utraty pakietów i desynchronizacji:

- **Identyfikatory próbek:** Pakiety zawierają unikalne numery oraz znaczniki czasu. Pozwala to na precyzyjny pomiar opóźnień w transmisji oraz ciągłe monitorowanie synchronizacji zegarów pomiędzy regulatorem a obiektem.
- **Utraty pakietów:** W przypadku braku nowej próbki w danym takcie, algorytm wykorzystuje ostatnią prawidłowo odebraną wartość. Zapewnia to ciągłość wyliczania uchybu oraz spójność wizualną na wykresach.
- **Diagnostyka wizualna:** Wydajność połączenia jest przeliczana w czasie rzeczywistym i prezentowana w GUI w formie liczbowej oraz trójkolorówek lampki statusu.

Historia Rozwoju Aplikacji

- Zaimplementowaliśmy klasy obsługujące asynchroniczne połączenia sieciowe pomiędzy klientem a serwerem. Zaprojektowaliśmy mechanizmy serializacji i deserializacji binarnej oraz tekstowej.
- Zabezpieczyliśmy aplikację przed błędami użytkownika oraz przypadkowymi kliknięciami w trakcie trwania symulacji. Wdrożyliśmy logikę automatycznego blokowania odpowiednich kontrolerek oznacza to, że po nawiązaniu połączenia interfejs automatycznie dostosowuje się do roli obiektu/regulatora. Obejmuje to m.in. blokadę nastaw regulatora po stronie obiektu oraz parametrów modelu ARX po stronie klienta regulatora.
- Wdrożyliśmy system automatycznego przesyłania pakietów pełnej konfiguracji między instancjami wywoływany po wykryciu modyfikacji dowolnego parametru w GUI.
- Zrealizowaliśmy dwa warianty sterowania czasem: tryb taktowania jednostronny, w którym symulacja obiektu działa na zegarze regulatora oraz tryb obustronny, który operuje na niezależnych zegarach regulatora i obiektu.
- Stworzyliśmy metodę, która w czasie trwania symulacji wyświetla parametry transmisji takie jak przepustowość, średnie opóźnienie oraz liczbę utraconych próbek. Zaimplementowaliśmy wizualną lampkę statusu symulacji oraz mechanizm awaryjny, który zapewnia bezpieczne zerwanie połączenia i

płynny powrót aplikacji do trybu stacjonarnego po wykryciu krytycznej utraty łączności.

Trudności Implementacyjne i Rozwiązania

– Opóźnienia w trybie obustronnym

Niezależne taktowanie regulatora i obiektu powodowało wrażliwość symulacji na zmienne opóźnienia sieciowe i straty pakietów. Rozwiązaliśmy to implementując zmienną pamięci ostatniej wartości, dzięki czemu węzeł nie czeka na zgubiony pakiet, lecz kontynuuje symulację wykorzystując ostatnio odebrane dane.

– Zerwanie połączenia

Konieczne było obsłużenie nagłego zerwania komunikacji z przyczyn zewnętrznych, co zrealizowano poprzez zatrzymanie timera sieciowego, czyszczenie buforu i wywołanie funkcji ustawTrybGUI (Stacjonarny). Dzięki temu kontrolki na interfejsie natychmiast się odblokowują, użytkownik dostaje wyraźny komunikat o zerwaniu transmisji, a symulacja może być płynnie kontynuowana.

– Problem ze zmianą trybu pracy

Po rozłączeniu aplikacja nie pozwalała na ponowne połączenie bez pełnego restartu. Problemem okazało się użycie metody disconnect(), która w Qt usuwa wszystkie powiązania sygnałów i slotów. Zmiana na disconnectFromHost() trwale rozwiązała ten błąd, ponieważ ona zrywała tylko połączenie sieciowe, a nie usuwała wewnętrzne powiązania sygnałów i slotów.

Podsumowanie Indywidualne

- Wojtek:** Nauczyłem się tworzyć i obsługiwać własny protokół komunikacyjny z systemem identyfikatorów, który potrafi przenosić zarówno pakiety JSON jak i ramki z binarne. Poznałem cykl życia połączeń sieciowych i zrozumiałem, jak projektować architekturę aplikacji odporną na brak danych sterujących. Projekt ten pomógł mi w uświadomieniu sobie jak ważny jest odpowiedni podział prac między poszczególnych członków zespołu.
- Paweł:** Nauczyłem się praktycznej implementacji komunikacji sieciowej, m. in. specyfiki protokołu TCP w architekturze klient-serwer oraz wyzwania związanych z przesyłaniem danych w czasie rzeczywistym, takich jak opóźnienia czy straty pakietów. Dodatkowo zrozumiałem jak ważna jest współpraca i systematyczność pracy. Projekt uświadomił mi, że w pracy

ważna jest nie tylko wiedza i umiejętności, ale również komunikacja i dobra atmosfera w zespole.

- **Krzysiek:** Nauczyłem się jak projektować protokół komunikacyjny na bazie TCP oraz jak projektować ramki. Nauczyłem się jak integrować własny kod z kodem napisanym przez wcześniejsze sekcje. Zrozumiałem z jakimi problemami wiąże się synchronizacja symulacji oraz jak sobie z nimi radzić.